evolvyn

ENGINEERING ARCHITECTURE / 2026

Harness engineering

Architecture, execution controls and evidence

How Evolvyn coordinates agents, tools, business evidence and human decisions. A repository-backed technical review prepared for engineering and product evaluation.

EVIDENCE	CONTROL	REVIEW
Source-bound context	Typed, bounded execution	Argument-bound approval

Prepared 25 October 2026 · Source baseline **bb82c1b**. Code-level assessment with fresh focused tests. Live provider authorization and production readiness require separate acceptance.

13 pages · Architecture diagrams · Implementation references

01 / EXECUTIVE PERSPECTIVE

The harness makes agent work reviewable

Evolvyn's harness is the application infrastructure around model inference: it controls which context an agent receives, which tools it can use, how long a run may continue, what is persisted, and when a human decision is required. A model response becomes a proposed artifact inside this system; it does not itself grant execution authority.

The main product loop is **evidence → interpretation → recommendation → approval → action receipt → outcome comparison**. Product Studio applies the same discipline to requirements, design review and approved source generation. Growth Insights applies it to business evidence and authorized social observations.

Three engineering decisions define the approach

- **Durability belongs in PostgreSQL.** Executions, checkpoints, approvals, tool registrations and business artifacts are stored records. In-process tasks perform work; they are not the durable queue.
- **Authority belongs in backend code.** Resource access, data-processing policy, tool risk, allowed capabilities and approval bindings are evaluated outside the model.
- **Claims need evidence.** Measured metrics, model interpretations, saved source, compiler receipts and live runtime behavior remain separate concepts.

The product benefit

A user can inspect an actual run, understand the node that failed, revisit completed drafts, and approve a specific action. Engineers can reason about bounded failure modes instead of an open-ended agent loop. The system retains its controls even though the Agent Harness and Governance screens have been removed from the current navigation.

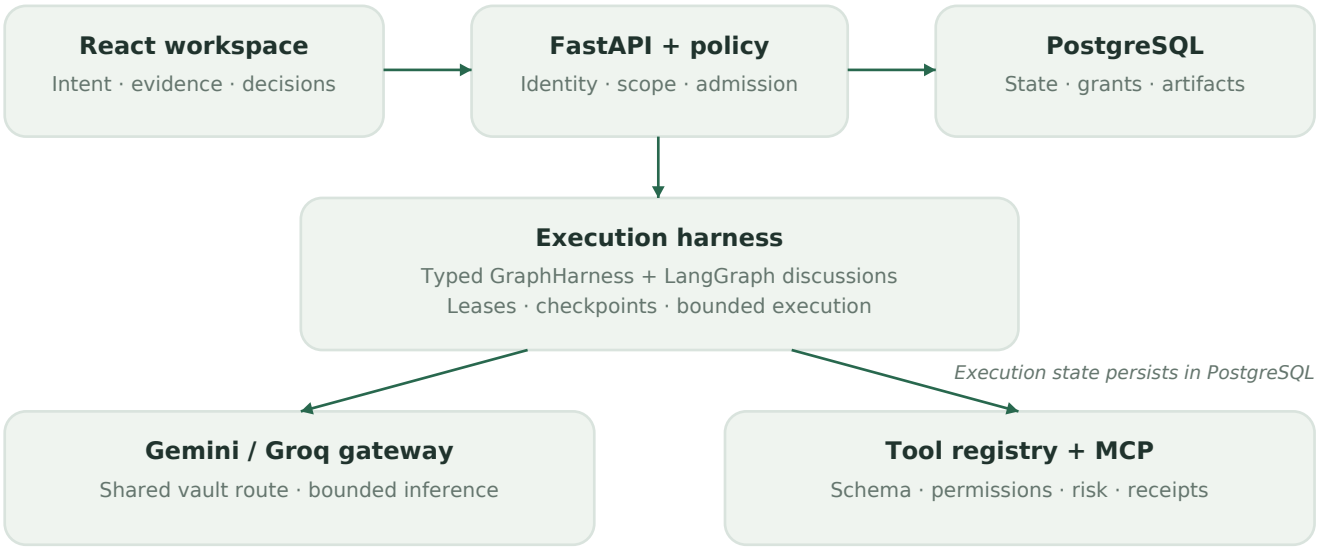
Scope of the assessment

The inspected code includes a custom declarative GraphHarness and a LangGraph discussion workflow. They reuse the execution queue, policy engine, model routing, registries and persistence. Current AI setup exposes Gemini and Groq with a shared workspace default; legacy provider/mode compatibility remains in backend schemas. Live provider availability depends on the deployed configuration.

Sources: [S01], [S02], [S03], [S04], [S08], [S09].

02 / SYSTEM ARCHITECTURE

One control layer, several workflows



Responsibilities and boundaries

- **React interface:** captures intent, uploads, configuration and decisions; renders persisted progress and artifacts. A browser control is not the authorization boundary.
- **FastAPI application:** authenticates the request, establishes organization/project scope and starts a persisted execution.
- **Execution services:** GraphHarness drives typed product/social/tool graphs. Discussion uses LangGraph specialist handoffs. Both use the existing worker-lease and state-update mechanisms.
- **Model and tool plane:** request-local vault routing supplies model credentials; registered tools execute through validated adapters. External connectors remain subject to data-processing policy.
- **PostgreSQL:** holds execution state, graph definitions, checkpoints, approvals, observations and artifacts under organization ownership.

Why this is a harness

The important abstraction is a controlled run with contracts and state transitions. Models propose text or structured outputs; deterministic code selects permitted execution paths and enforces the write boundary. External data, comments, tool responses and retrieved memories remain untrusted inputs.

The implementation uses JSON records for much of its domain persistence. This keeps current workflows cohesive, while high-volume deployments will need measured query/index design, retention controls and storage sizing. The four-worker limit is a deliberate current capacity bound, not a throughput benchmark.

Sources: [S01], [S02], [S03], [S04], [S05], [S06].

03 / ORCHESTRATION AND CONTRACTS

Typed graphs replace open-ended loops

A graph definition contains an entry node, typed nodes and edges, plus explicit budgets. Supported node types are **tool**, **agent**, **condition**, **approval** and **result**. Pydantic validation rejects duplicate node IDs, unknown edge endpoints, missing tool/predicate requirements and unreachable nodes before execution.

References resolve only to bounded prior node outputs. Branch conditions determine which edges activate; skipped branches propagate so joins can proceed. Cyclic nodes are identified, and repeated visits consume the same step and per-node visit budgets. This supports controlled revision loops without promising unconstrained autonomous planning.

CONTROL	BOUND	INTERPRETATION
Graph structure	32 nodes; 64 edges	Validated definition ceiling
Run budgets	32 steps; 16 tools; 2,048 tokens; 180 s	Defaults; maxima 100 / 64 / 8,192 / 600 s
Node controls	10 visits; 30 s timeout; 1 attempt	Defaults; max 100 visits / 120 s / 4 attempts
Checkpoint	600,000 serialized characters	Last 200 harness events retained
Policy interaction	Minimum of graph/policy where applied	Default policy: 16 tool calls; 300 s

Product Studio orchestration

Confirmed requirements feed the PRD, architecture, interface and reviewer phases selected for that run. Typed contract checks decide whether the reviewed design can proceed. A reviewer approval is design consistency feedback; the source-writing tool still pauses for an authorized human decision.

Discussion orchestration

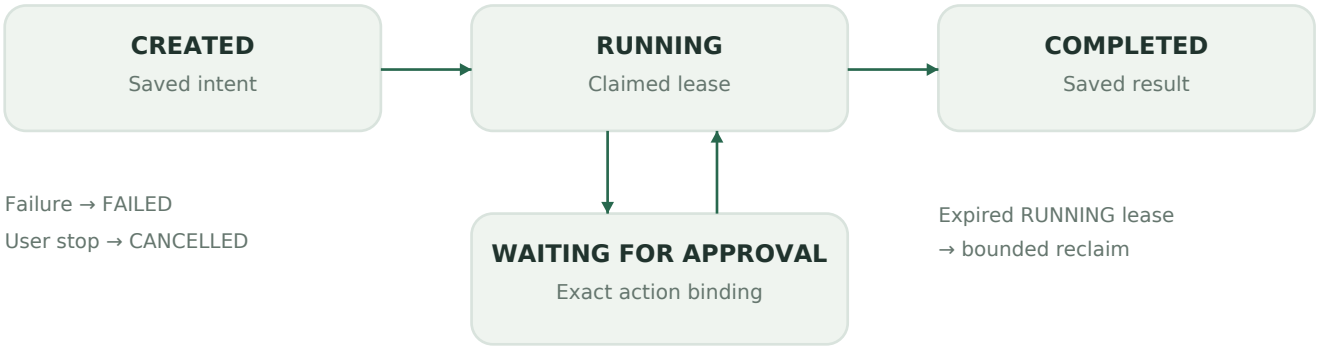
LangGraph sequences business analyst → marketing analyst → product manager → reviewer. Review issues can return to the product manager within a bounded number of rounds. Role output is cached in persisted discussion checkpoints. The workflow finishes in a human-review state before a decision is saved.

These are distinct orchestration implementations sharing platform controls. The product graph should not be described as a LangGraph graph merely because LangGraph is used elsewhere in the platform.

Sources: [S01], [S07], [S11].

04 / DURABLE EXECUTION

Recovery follows stored state



Admission and ownership

The API saves a CREATED execution and uses a conversation row lock to reject concurrent active runs in that conversation. A worker claims eligible work in a transaction. A PostgreSQL advisory lock serializes capacity admission, and at most four unexpired RUNNING leases are admitted across the queue.

Each claim receives a fresh worker token and a 60-second lease. The worker renews the lease every 10 seconds. State updates check the worker token under a row lock; a stale worker cannot overwrite a newer claimant. Terminal state guards prevent late progress updates from resurrecting a cancelled run.

Checkpoint-aware recovery

Graph checkpoints retain completed node outputs, active/queued work, visit counts, budget reservations and pending approvals. Expired work can be reclaimed, with a maximum of two worker claims before terminal recovery failure. Graceful shutdown requeues unfinished work rather than depending on a browser connection.

Retries are selective. Transport failures, timeouts and selected transient HTTP statuses may retry within configured limits. Policy denial or invalid contracts do not become permission to retry indefinitely. An interrupted non-idempotent write is treated as uncertain and requires reconciliation before another attempt.

Delivery semantics

This is bounded recovery, not a universal exactly-once guarantee. Checkpoints and idempotency keys reduce duplication risk; a remote system can complete a write before its response is lost. External receipts or reconciliation are still required. SSE exposes persisted updates and allows a returning client to reconnect; it does not make the browser the job owner.

Sources: [S01], [S02], [S03].

05 / AUTHORIZATION AND HUMAN DECISIONS

Approval binds an action, not a conversation

The policy engine combines organization ownership, current membership, role, project access, tool allow/deny lists, risk ceilings and external-processing settings. Viewer and discussion-only roles cannot acquire execution or approval authority by placing instructions in a prompt. Owner/admin checks protect registry, policy and vault management.

The write path

1. Resolve arguments from validated prior outputs.
2. Resolve the current registered tool, including server/configuration metadata.
3. Evaluate the agent capability set, argument schema, resource scope and organization policy.
4. For a non-read action, persist a pending approval containing the exact arguments and a digest binding.
5. Accept a decision only while that execution is awaiting approval and its binding remains current.
6. Recheck the current tool configuration, consume the grant and execute through the policy-enforced adapter.

The graph binding combines tool name, arguments and a specification fingerprint, plus the graph step. Separate tool approvals also bind the registered specification and move transactionally out of WAITING_FOR_APPROVAL so a consumed approval cannot be replayed. Changed risk, server configuration or credential-version metadata invalidates the old specification binding.

Data and credential boundaries

Restricted project resources are checked through linked conversations and artifacts. External model/MCP processing is denied when organization policy or a local-only project requires it. Removing a member can invalidate ongoing execution access. Credentials are resolved from scoped vault references and passed through provider headers, rather than inserted into prompts or tool arguments.

Practical boundary

Approval proves an authorized decision about a specific requested action. It does not establish that the action is useful, safe in every business context, or successfully completed remotely. Risk classification and adapter behavior still require operator review and trustworthy implementation.

Sources: [S01], [S04], [S05], [S06], [S08].

06 / CONTEXT, MODELS AND MEMORY

Evidence enters; authority stays outside

The context pipeline combines authorized source excerpts, calculated business observations, bounded conversation history, previous tool outputs and relevant reviewed memory. Citations tie interpretations to supplied evidence. Numerical business calculations remain deterministic; model text must not replace missing source data with estimated facts.

Shared Gemini/Groq routing

Saving a supported key validates its authenticated model catalog and a small JSON completion before establishing a shared workspace default. All agents use that default; current UI setup has no per-agent assignment. A failed replacement preserves the previous credential and default. An unavailable provider fails visibly rather than silently switching providers.

Request-local model scope chooses the organization and role. Vault secrets are decrypted for the matching provider at the service boundary. Native adapters enforce fixed destinations, timeouts and response-size limits, check output usability, and return provider-reported usage when available. Catalog verification is point-in-time evidence, not a guarantee of future quota or compatibility.

Reviewed long-term memory

Memory is proposed as DRAFT and accepted or rejected through an explicit review. Records include category, provenance, project scope, agent scope and expiry. Recall filters accepted, unexpired entries within the matching project and shared/role scope; ranking uses lexical overlap. It is not embedding-based semantic retrieval or model fine-tuning.

Prompt-injection posture

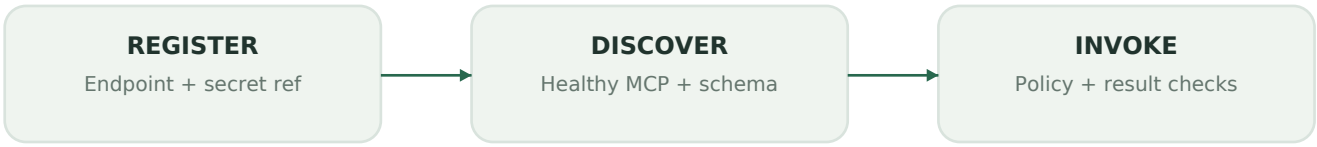
Uploaded documents, social comments, MCP responses and memory are treated as data. Schema validation, tool capabilities, secret-pattern checks and deterministic policy limit the consequences of misleading text. These are layered controls; they do not prove prompt injection has been eliminated. Broad red-team coverage and domain-specific adversarial evaluation remain release work.

Sources: [S04], [S06], [S08], [S09], [S10], [S14].

07 / MCP AND TOOL INTEGRATION

Connectivity is a validated execution path

MCP extends the tool plane. Registering a server supplies an endpoint/profile and credential reference; successful health discovery supplies tool schemas. A separate tool registration maps a discovered remote tool into the platform's persistent capability/entity catalog. Presence in the gallery alone is not a successful live connection.



Non-read tool → exact-argument approval before execution

Contract enforcement

Built-in tools have typed input/output models, declared risk, permission and timeout. Remote tools require validated arguments against the discovered JSON Schema, bounded MCP execution and any advertised protocol output contract. Remote output remains untrusted even when it conforms to its schema.

Public HTTPS MCP registration requires an explicitly approved hostname and rejects unsafe destinations and credential-bearing URLs. Secrets use scoped vault references or supported private runtime bindings. Host approval does not authorize data export from a restricted project, and OAuth sign-in does not grant analytics permissions for other services.

A concrete standalone server

The repository includes a separate authenticated Streamable HTTP MCP starter with two read-only tools: calculate_conversion and compare_conversion_rates. Its health endpoint is public; MCP calls require a strong bearer token. Host/origin validation, request-size bounds and a non-root Docker image support controlled deployment.

Example invocation: visits=200 and paid=30 produces a 15% conversion rate. This is a deterministic calculation using supplied counts, not a live sales integration. The starter demonstrates protocol registration and tool invocation; it does not establish continuous social monitoring or arbitrary third-party tool safety.

Extension rule

A user can register a compatible authorized endpoint and discovered tool. Registering a REST URL or uploading arbitrary handler code does not create an MCP server. Native adapters require reviewed implementation and deployment.

Sources: [S05], [S06], [S12], [S13], [S15].

08 / PRODUCT STUDIO EXECUTION BOUNDARY

Source generation is an artifact workflow

The Product Studio harness starts with versioned, confirmed requirements. Requirements revisions are checked when planning and writing artifacts. PRD, architecture, interface and review outputs are proposals until their contracts are validated and the user approves the source implementation action.

The approved source pipeline

1. Request a model-authored manifest of no more than 16 text files.
2. Validate unique safe relative paths and required React + FastAPI/Express entry points; reject fabricated lockfiles.
3. If the manifest is malformed, request one corrected manifest within the existing budget. Never weaken validation to accept it.
4. Generate one file per model call and persist completed files individually. A failed run retains completed files and its failure category.
5. Parse supported contracts: JSON, Python syntax and restricted Compose privileges/volumes; calculate a source digest.
6. Request frontend compilation from the operator-configured private compiler and retain its receipt.

Default source limits are 4,096 output tokens per file and 65,536 reserved output tokens per approved build. The implementation caps generation at 34 model calls and 16 files. Source generation has its own counters inside an approved tool invocation; these should not be confused with the outer graph's model-token counter. Cost accounting needs both layers.

What the result proves

Saved files prove persisted source. A passed compiler receipt proves the restricted frontend compiled. The scoped data prototype demonstrates platform-managed data interaction. Neither proves the generated backend is running, authentication is correct, business requirements are complete or a public application is deployed.

The API does not execute arbitrary generated application code. Backend runtime tests, isolated live previews, targeted repair and deployment require a separate execution boundary. If no compiler is configured, the receipt is unavailable; it is not converted into a successful compilation claim.

Sources: [S01], [S11], [S16], [S17].

09 / GROWTH INSIGHTS AND MONITORING

Business intelligence starts with source quality

Growth Insights connects first-party uploads, deterministic metrics, authorized observations and specialist interpretation. Research/business reasoning supports a decision; it does not convert sampled post engagement into revenue, unique customers or causal proof of growth.

Social collection through the harness

A social source binds a conversation, provider account and registered read-only MCP observation tool. The saved specification fingerprint detects tool or credential configuration changes. The source creates a bounded observation graph, which can be run manually or scheduled through the existing persisted execution system.

Collection verifies the account/provider identity, observation schema, capture freshness and current configuration before saving a snapshot. Snapshot IDs are derived from organization, execution and source, making repeated processing of the same collection identifiable. The specialist review can reference the saved snapshot and its coverage limitations.

Monitoring semantics

Recurring schedules and upload-event automations enqueue work; they are not evidence of a healthy continuous connection. API permissions, token expiry, server sleep, provider quotas and missing intervals can all interrupt observation. Operators must verify successful receipts and freshness instead of relying on a connected badge.

Conservative forecasting

The implemented social forecast requires at least 29 comparable daily snapshots for 28 daily interaction increments. It checks source freshness, stable account/post cohort, near-daily spacing, configuration consistency and nondecreasing counters. A seven-day rolling mean is compared with a last-day baseline over a recent backtest; a worse result is suppressed.

The forecast concerns new interactions on the same observed posts. Its range uses recent absolute backtest errors, not a calibrated probability interval. Revenue, demand and customer-level predictions remain unsupported by this evidence. General growth reports suppress forecasts when comparable history and backtesting have not been established.

Sources: [S18], [S19], [S20], [S21].

10 / VERIFICATION AND FAILURE HANDLING

Test the controls, not only the happy path

SUITE	RESULT	WHAT IT EXERCISES
Graph harness	7 tests	Approval, branches/joins, loops, retries and budgets
Worker recovery	4 tests	Expired leases, stale fencing, attempt cap, data policy
Registry + governance	9 tests	Discovery, schemas, approvals, tenant and network limits
Project isolation	1 test	Restricted linked-resource visibility
Source generation	20 tests	Persistence, cancellation, unsafe paths and manifest repair

The focused suite was rerun for this document on 25 October 2026. Tests use isolated PostgreSQL schemas and controlled boundaries where a real external provider is unavailable. They substantiate the inspected controls, not every platform feature or the health of the public deployment.

Failure cases with useful engineering meaning

- **Changed approval context:** an altered tool/risk configuration makes the previous grant stale; the caller must request approval again.
- **Worker replacement:** a stale worker token cannot overwrite the new lease holder's execution state.
- **Unbounded loop attempt:** graph iteration/visit budgets stop repeated work rather than allowing an indefinite model cycle.
- **Unsafe remote endpoint:** credential URLs, private-network destinations and cross-organization credential references are rejected.
- **Invalid source manifest:** one correction is allowed; repeated invalid output fails without saving fabricated files.
- **Partial source failure:** completed files remain inspectable, while unfinished files are not represented as generated.

Evidence to inspect in a demonstration

Show an execution ID and the saved graph/checkpoint. Trigger an approval, alter or reject its binding, and show the actual denial. Inspect a tool trace and saved result. Disconnect a provider or omit the compiler to demonstrate an explicit failed/unavailable state. End with a source-linked outcome comparison rather than a claimed business improvement.

No live Gemini/Groq key or deployed OAuth grant was used in this focused rerun. This document does not infer successful live provider authorization from mocked HTTP tests. Historical test totals in older documentation are excluded from this verification statement.

Sources: [T01], [T02], [T03], [T04], [T05].

11 / TRADEOFFS AND PRODUCTION PATH

A credible foundation, with explicit next gates

What the implementation does well

The harness keeps persistence, model routing, tool execution, approvals and business evidence connected. It has inspectable checkpoints, bounded work, revocable credentials, scoped memory and explicit compiler receipts. Those choices make failures diagnosable and consequential actions reviewable.

Current tradeoffs

- **PostgreSQL-backed jobs:** simple infrastructure and transactional state, with a four-worker global admission limit. Measure contention, backlog age and JSON query costs before scaling.
- **Two orchestration paths:** custom typed graphs and LangGraph discussions share services but have different checkpoint structures. Documentation and evaluation must preserve that distinction.
- **Reserved tokens:** bounded reservations constrain attempts but are not actual token bills. Reconcile provider usage, source-builder calls and estimates for operational cost reporting.
- **Registered tool risk:** policy enforces the declared risk, while correct classification and adapter behavior remain essential. A misleading read classification is a security concern.
- **Partial execution guarantees:** idempotency and fencing address known replay risks, but cannot prove external exactly-once effects.

Release gates to prioritize

1. Validate live Gemini/Groq routes and every supported OAuth flow with the deployed origins, permissions, expiry and revocation behavior.
2. Run backup/restore drills, secret-key recovery, tenant-isolation and session/recovery security reviews.
3. Add workload measurements and alerts for queue lag, lease churn, provider latency/errors, approval backlog and observation staleness.
4. Establish repeatable task-quality and adversarial evaluations before changing prompts, models or tool capabilities.
5. Add a separate isolated generated-app runtime with resource/network restrictions and domain/security acceptance tests before promising live previews or deployment.

The strongest technical presentation is a reproducible explanation of the controls and their limits. A polished interface, a passing unit suite or an agent's own review is not a production-readiness certificate.

Sources: [S01]–[S21], [T01]–[T05].

12 / SOURCE AND EVIDENCE INDEX

Follow each claim into the repository

References below link to the inspected GitHub revision. The source baseline fixes the meaning of this document even if the repository changes later.

REF	IMPLEMENTATION / EVIDENCE	REPOSITORY FILE
S01	Graph definitions, execution and approvals	backend/graph_harness.py
S02	Worker claims, capacity and leases	backend/execution_queue.py
S03	State fencing, lease renewal and live events	backend/app.py
S04	Deterministic resource, risk and data policy	backend/policy.py
S05	Persistent adapters and approval digests	backend/tool_plugins.py
S06	Typed built-in tool invocation	backend/registry.py
S07	LangGraph discussion and revision loop	backend/discussion.py
S08	Encrypted credentials and shared routes	backend/vault.py
S09	Catalog and JSON compatibility checks	backend/cloud_models.py
S10	Reviewed, scoped lexical memory	backend/agent_memory.py
S11	Product specialist graph and revisions	backend/product_agents.py
S12	MCP registration, sessions and discovery	backend/mcp_registry.py
S13	Bundled MCP client validation	backend/mcp_gateway.py
S14	Request-local model/credential scope	backend/model_access.py
S15	Authenticated standalone MCP starter	backend/custom_mcp.py
S16	Manifest and file-by-file source builder	backend/model_source.py
S17	Private compiler request and receipts	backend/source_compiler.py
S18	Social source binding and snapshot graph	backend/social_growth.py
S19	Social observation and forecast contracts	backend/social_contract.py
S20	Evidence-led growth analysis	backend/growth.py
S21	Persisted upload-event automations	backend/automations.py
T01	Graph branches, joins, budgets and retries	tests/test_graph_harness.py
T02	Recovery and stale-worker fencing	tests/test_recovery.py
T03	Registry, approval and remote boundaries	tests/test_registry_governance.py
T04	Restricted project resource isolation	tests/test_project_policy.py
T05	Manifest recovery and retained source	tests/test_model_source.py

Verification record

Command: `.venv/bin/python -m pytest -q tests/test_graph_harness.py tests/test_recovery.py tests/test_registry_governance.py tests/test_project_policy.py tests/test_model_source.py`

41 passed in 37.47 seconds. One Starlette/AnyIO dependency deprecation warning. No live provider acceptance is inferred.

Assessment date: 25 October 2026. No deployment credentials, customer data, OAuth tokens or model API keys are included in this document.